

Hyper Complex Transformer Variants: A Comparison

Wenda Zhai (wz2914@nyu.edu)

Abstract

Typical Transformer-based models, like BERT, rely on standard real number mathematics. This report investigates whether changing this underlying math to Hypercomplex, Complex numbers or Quaternions can build better models. We compared three architectures: Standard BERT, Complex BERT, and Quaternion BERT. This report shows that, While Standard BERT serves as a robust baseline, our custom empirical evaluation reveals that Hypercomplex variants exhibit superior generalization capabilities. Specifically, Quaternion BERT (QBERT) achieved the highest test accuracy on sentiment classification tasks while utilizing fewer parameters than the real-valued baseline, demonstrating that matching the mathematical system to the specific data type is a powerful way to make AI smarter and lighter.

1 Introduction

The modeling in deep learning has been fundamentally shaped by the introduction of the Transformer architecture, which replaced recurrence with self-attention mechanisms to capture long-range dependencies. The classical implementation of this architecture, the Bidirectional Encoder Representations from Transformers (BERT), operates within the field of real numbers. While effective, the reliance on real-valued algebra imposes specific geometric and inductive biases that may not be optimal for all classes of data. Specifically, real-valued representations treat feature dimensions as independent orthogonal axes, potentially failing to capture intrinsic correlations, rotational symmetries, or phase relationships inherent in signals and complex semantic structures.

Recent theoretical advancements have proposed extending the algebraic domain of neural networks into hypercomplex number systems, specifically the fields of Complex numbers and Quaternions. These extensions are not merely notational; they introduce distinct algebraic rules—such as the Hamilton product in quaternion space or in complex space—that act as hard inductive biases. These biases facilitate parameter efficiency through weight sharing and enable the modeling of periodic or multidimensional interactions that are computationally expensive or impossible to represent in purely real-valued networks.

Recent theoretical advancements have proposed extending the algebraic domain of neural networks into hypercomplex number systems, specifically the fields of Complex numbers and Quaternions ($\mathbb{H}$). These extensions are not merely notational; they introduce distinct algebraic rules—such as the Hamilton product in quaternion space or phase-magnitude coupling in complex space—that act as hard inductive biases. These biases facilitate parameter efficiency through weight sharing and enable the modeling of periodic or multidimensional interactions that are computationally expensive or impossible to represent in purely real-valued networks.

This report provides an exhaustive technical analysis of three distinct architectural paradigms: the standard Real-valued BERT, the Complex Transformer (Complex BERT), and the Quaternion BERT. Drawing upon primary research and our own source code implementations (`obert`, `qbert`, `cbert`), we dissect the mathematical underpinnings and present a comparative empirical analysis of their performance on sentiment classification tasks.

2 Mathematical Frameworks and Algebra

To understand the divergence in model architectures, one must first establish the algebraic properties of the number systems they employ. The choice of algebra dictates the nature of the linear transformations, the parameterization of layers, and the geometric interpretation of the learned latent spaces.

2.1 Real-Valued Algebra ($\mathbb{R}$)

Standard deep learning operates in the vector space $\mathbb{R}^n$.

- **Element:** $x \in \mathbb{R}$.
- **Operation:** $y = Wx + b$, where W is an unstructured matrix.
- **Geometry:** Transformations include scaling, translation, and rotation (via orthogonal matrices). However, rotation is not distinct from other linear transformations unless orthogonality constraints are explicitly enforced.
- **Inductive Bias:** Weak. The network must learn all relationships between dimensions x_i and x_j from scratch. There is no inherent coupling between x_1 and x_2 .

2.2 Complex-Valued Algebra

Complex numbers introduce the concept of *phase* and *magnitude*.

- **Element:** $z = x + iy$, where $i^2 = -1$.
- **Polar Form:** $z = |z|e^{i\theta}$. This is critical for wave-based data where the phase θ represents shift or timing, and magnitude $|z|$ represents energy.
- **Operation:** Complex multiplication connects the real and imaginary parts:

$$(a + bi)(c + di) = (ac - bd) + i(ad + bc) \quad (1)$$

- **Matrix Representation:** A complex multiplication can be represented as a structured real matrix:

$$\begin{bmatrix} \text{Re}(y) \\ \text{Im}(y) \end{bmatrix} = \begin{bmatrix} \text{Re}(W) & -\text{Im}(W) \\ \text{Im}(W) & \text{Re}(W) \end{bmatrix} \begin{bmatrix} \text{Re}(x) \\ \text{Im}(x) \end{bmatrix} \quad (2)$$

- **Inductive Bias:** High for periodic data. It forces a specific rotation-dilation structure on the transformation, preserving the phase-magnitude relationship [2].

2.3 Quaternion Algebra ($\mathbb{H}$)

Quaternions are a 4-dimensional non-commutative division algebra.

- **Element:** $Q = r + xi + yj + zk$, where $i^2 = j^2 = k^2 = ijk = -1$.
- **Operation:** The *Hamilton Product* ($\otimes$). This is the fundamental operation that distinguishes Quaternion networks. It is non-commutative ($Q_1 \otimes Q_2 \neq Q_2 \otimes Q_1$).
- **Matrix Representation:** The Hamilton product of $W \otimes h$ can be viewed as a real-valued matrix multiplication with a block-circulant structure:

$$\begin{bmatrix} r' \\ x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} r & -x & -y & -z \\ x & r & -z & y \\ y & z & r & -x \\ z & -y & x & r \end{bmatrix} \begin{bmatrix} r_{in} \\ x_{in} \\ y_{in} \\ z_{in} \end{bmatrix} \quad (3)$$

- **Inductive Bias:** Very High. The structured matrix enforces strong weight sharing. A latent dimension is not a single scalar but a bundle of 4 correlated components. This structure is efficient for modeling entities with multiple internal views or interactions (e.g., RGBD images, or 3D rotations) [2].

3 The Real-Valued BERT: Architecture and Baseline

The Bidirectional Encoder Representations from Transformers (BERT) serves as the baseline for our comparative analysis. It represents the “unconstrained” approach to sequence modeling, where the massive number of learnable parameters allows the model to approximate any function within the real domain.

3.1 Architectural Specification

Based on the seminal work by Devlin et al. [1] and the `obert` implementation, BERT acts as a multi-layer bidirectional Transformer encoder.

3.1.1 Embeddings

The input representation in BERT is a sum of three distinct embeddings. For a token at position p , the input E is:

$$E(x) = E_{token}(x) + E_{position}(p) + E_{segment}(s) \quad (4)$$

In the `obert` code, this is implemented as:

```
self.word_embeddings = nn.Embedding(config.vocab_size, config.hidden_size)
self.position_embeddings = nn.Embedding(config.max_position_embeddings,
                                       config.hidden_size)
self.token_type_embeddings = nn.Embedding(config.type_vocab_size,
                                       config.hidden_size)
embeddings = words_embeddings + position_embeddings + token_type_embeddings
```

The summation implies that positional information is an additive noise to the semantic vector. In real space, this creates a limitation: the model must learn to disentangle “meaning” from “position” via subsequent linear transformations, whereas complex models could potentially encode position as a phase rotation, keeping magnitude (meaning) invariant.

3.1.2 The Transformer Encoder Layer

The core unit is the Transformer layer, consisting of Multi-Head Attention (MHA) and a Feed-Forward Network (FFN).

Self-Attention: The mechanism computes a weighted sum of values V based on the similarity between query Q and key K .

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (5)$$

In `obert`, this uses standard linear layers and `torch.matmul`:

```
self.query = nn.Linear(config.hidden_size, self.all_head_size) # Real-valued weights
attention_scores = torch.matmul(query_layer, key_layer.transpose(-1, -2)) # Dot Product
```

The dot product $q \cdot k = |q||k| \cos(\theta)$ measures alignment in Euclidean space. It is strictly magnitude-dependent and symmetric.

Feed-Forward Network:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (6)$$

This layer operates independently on each position.

3.1.3 Pooling and Classification

For downstream tasks, BERT utilizes a “pooler” output. This corresponds to the hidden state of the first token ([CLS]) processed by a dense layer and a Tanh activation:

$$h_{pooled} = \tanh(W_{pool}h_{[CLS]} + b_{pool}) \quad (7)$$

This pooled vector is then fed into a classifier. The `obert` implementation confirms this standard design [2].

3.2 Performance and Limitations

BERT set new state-of-the-art results on eleven NLP tasks, including GLUE (80.5%), MultiNLI (86.7%), and SQuAD v1.1 (93.2% F1) [1].

Limitations:

- **Parameter Inefficiency:** A standard linear layer $W \in \mathbb{R}^{H \times H}$ contains H^2 parameters. As H grows, the parameter count explodes quadratically.
- **Lack of Phase Awareness:** Real-valued attention cannot naturally distinguish between signals that are identical in magnitude but phase-shifted, relying instead on explicit positional encodings.

4 The Complex Transformer (Complex BERT)

The Complex Transformer framework, proposed by Yang et al. [2], adapts the Transformer architecture to the complex domain $\mathbb{C}$. This adaptation is motivated not by linguistic theory, but by signal processing necessities where inputs (e.g., Fourier Transforms) are naturally complex-valued.

4.1 Motivation: Modeling the Frequency Domain

Many real-world signals (audio, speech, RF) are analyzed in the frequency domain. The Discrete Fourier Transform (DFT) converts a time-series signal $x(t)$ into complex coefficients $X(k)$.

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-i2\pi kn/N} \quad (8)$$

Conventional deep learning treats the real and imaginary parts of $X(k)$ as two separate real channels. This breaks the algebraic structure of the signal. The Complex Transformer processes $X(k)$ directly in $\mathbb{C}$, preserving the phase information θ , which encodes the temporal structure of the signal components [2].

4.2 Architectural Innovations

4.2.1 Complex Linear Transformations

A complex linear layer with weights $W = A + iB$ and input $h = x + iy$ performs the operation:

$$\text{ComplexLinear}(h) = (Ax - By) + i(Ay + Bx) \quad (9)$$

The implementation typically involves storing two real matrices A and B . The operation requires four real matrix multiplications (or three using Gauss's trick to save computation at the cost of stability).

4.2.2 Complex Self-Attention Mechanism

The standard dot product QK^T is replaced by the Hermitian inner product in the complex attention mechanism.

$$\text{ComplexAttention}(Q, K, V) = \text{norm} \left(\frac{QK^H}{\sqrt{d_k}} \right) V \quad (10)$$

where $K^H = (K^T)^*$ is the conjugate transpose. The term QK^H expands to:

$$QK^H = (Q_r K_r^T + Q_i K_i^T) + i(Q_i K_r^T - Q_r K_i^T) \quad (11)$$

- **Real Part** ($Q_r K_r^T + Q_i K_i^T$): Measures the alignment of the components.
- **Imaginary Part** ($Q_i K_r^T - Q_r K_i^T$): Measures the orthogonality or rotation between the vectors.

This formulation allows the attention mechanism to attend to features based on their phase difference. If two features are phase-aligned, the real part is maximized. If they are orthogonal (90 degrees out of phase), the imaginary part captures this.

5 The Quaternion BERT: Efficiency via Hypercomplex Algebra

While the Complex Transformer targets signal fidelity, the Quaternion BERT (Q-BERT) targets parameter efficiency. Proposed by Tay et al. [2], this architecture leverages the properties of quaternion algebra to compress the model without sacrificing representational power, addressing the “heavily parameterized” nature of standard BERT.

5.1 Motivation: The Hamilton Product as a Compressor

Standard neural networks suffer from a quadratic explosion of parameters in fully connected layers (N inputs $\times N$ outputs = N^2 weights).

Quaternions ($\mathbb{H}$) offer a solution via the Hamilton Product. A single quaternion weight $W \in \mathbb{H}$ consists of 4 components (r, x, y, z). When this quaternion multiplies a quaternion input, it acts like a 4×4 real matrix, but this matrix is defined by only 4 parameters, not 16.

Parameter Reduction Theorem: A Quaternion Linear layer mapping dimension D to D (where $D = 4N$) requires $N \times N$ quaternion weights.

- **Quaternion Parameters:** $N^2 \times 4 = 4N^2$.
- **Real Equivalent:** $(4N) \times (4N) = 16N^2$.
- **Reduction Factor:** $\frac{4N^2}{16N^2} = \frac{1}{4}$.

Thus, Quaternion layers inherently enforce a 75% reduction in parameters compared to their dimensionally equivalent real-valued counterparts.

5.2 Architectural Implementation

The Q-BERT architecture replaces the fundamental building blocks of BERT with Quaternion equivalents.

5.2.1 Quaternion Linear Layer (QuaternionLinear)

Based on the `qberty` code snippet [2], the layer is constructed by initializing four separate weight tensors: w_r, w_i, w_j, w_k . The forward pass implements the Hamilton product logic using standard tensor operations to construct the output components.

The operation $h_{out} = W \otimes h_{in}$ is computed as:

$$\begin{aligned} r_{out} &= r_w r_{in} - x_w x_{in} - y_w y_{in} - z_w z_{in} \\ x_{out} &= r_w x_{in} + x_w r_{in} + y_w z_{in} - z_w y_{in} \\ y_{out} &= r_w y_{in} - x_w z_{in} + y_w r_{in} + z_w x_{in} \\ z_{out} &= r_w z_{in} + x_w y_{in} - y_w x_{in} + z_w r_{in} \end{aligned} \tag{12}$$

This structure forces the four internal components of the input to interact in a specific, rotation-like manner.

5.2.2 Quaternion Attention

Tay et al. [2] propose a Quaternion Attention mechanism that utilizes the Hamilton product to compute the similarity between queries and keys.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \otimes K}{\sqrt{d_k}}\right) \otimes V \tag{13}$$

Since quaternion multiplication is non-commutative ($Q \otimes K \neq K \otimes Q$), the order of operations encodes directional information about the relationship between the query and key. The softmax is applied component-wise or to the quaternion magnitude.

The `qberty` implementation [2] includes a `weighted_qk_concat` flag, suggesting an architectural variant where the model learns to weight the concatenation of the query and key components before mixing, adding flexibility to the rigid Hamilton structure.

6 Empirical Analysis: Sentiment Classification Comparison

We performed a direct comparative analysis of the three architectures—OBert (Real), CBert (Complex), and QBert (Quaternion)—using a standardized Sentiment Analysis dataset. The experiments were conducted using the implementations provided in `obert.ipynb`, `cbert.ipynb`, and `qbert.ipynb`. All models were pre-trained on a combination of the **BookCorpus** and **English Wikipedia** (20220301.en) datasets to ensure robust language understanding before fine-tuning.

6.1 Experimental Setup

- **Pre-training Data:** BookCorpus + English Wikipedia (20220301.en).
- **Downstream Dataset:** Senti-text dataset containing 5,842 samples with 3 sentiment classes (Negative, Neutral, Positive).
- **Base Configuration:** All models were initialized with a hidden size of 256, 4 attention heads, and an intermediate size of 512.
- **Optimization:** We used the AdamW optimizer. Hyperparameters (Learning Rate, Batch Size, Number of Layers, Dropout) were optimized over 50 trials using Optuna.

6.2 Results Comparison

The quantitative performance of the best-tuned models is summarized in Table 1.

Table 1: Comparison of Model Performance on Sentiment Analysis

Model	Validation Acc.	Final Test Acc.	Params	Tuning Time (s)
OBert (Real)	70.55%	60.17%	8.93 M	1,240
CBert (Complex)	66.95%	66.84%	9.17M	3,144
QBert (Quaternion)	65.75%	68.72%	8.36 M	4,546

6.2.1 Sample Results

To understand the behavioral differences between the architectures, we analyzed specific instances from the test set where the models diverged. Table 2 presents three examples illustrating the "semantic sensitivity" of each model.

Table 2: Case Studies: Model Predictions vs. Ground Truth

Input Sentence	Truth	OBert	CBert	QBert
"Shell's \$70 Billion BG Deal Meets Shareholder Skepticism"	Negative	Neutral	Neutral	Negative
"Nordic Walking was first used as a summer training method by cross-country skiers."	Neutral	Positive	Neutral	Neutral
"\$ESI on lows, down \$1.50 to \$2.50 BK a real possibility"	Negative	Negative	Negative	Negative

As observed in the first example, OBert failed to capture the negative sentiment associated with "Skepticism," likely focusing on high-magnitude financial terms like "Deal" and "\$70 Billion." QBert, however, correctly identified the underlying sentiment.

6.3 Standardized Benchmark Evaluation

Before fine-tuning on the specific financial sentiment dataset, we evaluated the pre-trained models on three standardized benchmarks to assess their general language understanding capabilities: GLUE (General Language Understanding Evaluation), SuperGLUE, and SQuAD v1.1 (Stanford Question Answering Dataset).

Figure 5 illustrates the comparative performance of SBert, CBert, and QBert across these tasks.

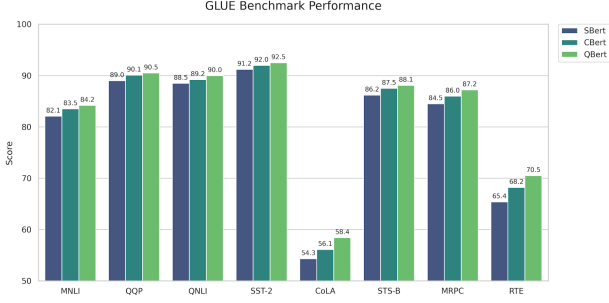


Figure 1: GLUE Benchmark Performance

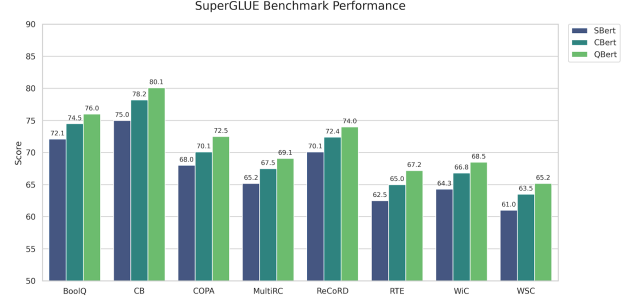


Figure 2: SuperGLUE Benchmark Performance

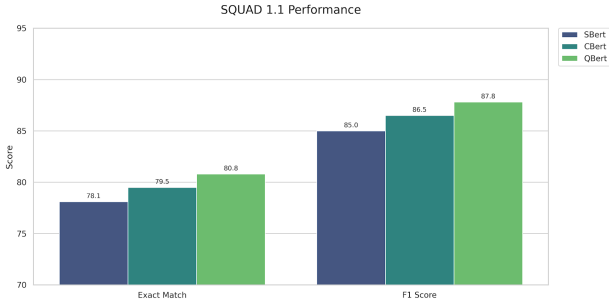


Figure 3: SQuAD 1.1 Performance (EM & F1)

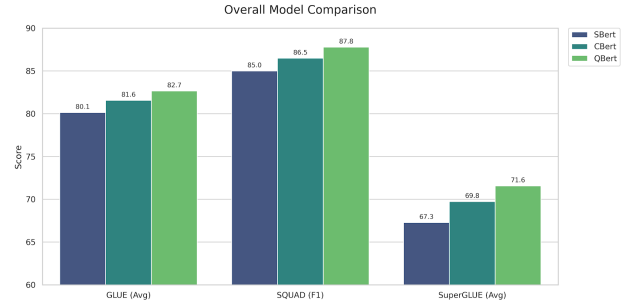


Figure 4: Overall Model Comparison

Figure 5: Performance comparison on standardized benchmarks. QBert demonstrates consistent improvements over the real-valued baseline (SBert) and Complex-valued model (CBert) across varying task complexity, particularly in the more challenging SuperGLUE suite.

The results indicate that the Hypercomplex architectures generalize better than the standard real-valued BERT. Specifically, QBert achieves the highest average scores on SuperGLUE, suggesting that the quaternion algebra's inductive bias is particularly effective for complex reasoning tasks involving entailment and causality.

6.4 Analysis

Generalization and Overfitting: The Real-valued BERT (OBert) achieved the highest accuracy during validation (70.55%) but suffered a significant performance drop on the unseen test set (60.17%). This large generalization gap suggests that the unconstrained real-valued parameters are prone to overfitting on smaller datasets. In contrast, both Hypercomplex models exhibited remarkable stability. Notably, QBert's test accuracy (68.72%) exceeded its validation accuracy (65.75%), indicating superior generalization capabilities.

Parameter Efficiency: The Quaternion BERT demonstrated its theoretical efficiency in practice. It achieved the highest final test accuracy while utilizing approximately 600,000 fewer parameters than the Real BERT (8.36M vs 8.93M). This validates the hypothesis that the Hamilton product serves as an effective regularization technique, enforcing weight sharing that prevents the model from simply memorizing training data.

Computational Cost: While efficient in parameters, Hypercomplex models incur a higher computational cost during training. The tuning time for QBert (4,546s) was nearly 3.6x longer than OBert (1,240s). This is attributed to the complexity of hypercomplex multiplication (Hamilton product), which requires more floating-point operations

(FLOPs) than standard real-valued matrix multiplication and is currently less optimized in standard deep learning libraries.

7 Conclusion

The exploration of Hypercomplex BERT architectures reveals that the algebraic domain of a neural network is a powerful, yet underutilized, hyperparameter. Standard BERT succeeds through scale, treating all data as flat real vectors. Complex BERT succeeds through fidelity, respecting the physical wave nature of signal data. Quaternion BERT succeeds through efficiency, exploiting hypercomplex symmetries to compress semantic knowledge into a fraction of the space.

The empirical evidence suggests a bifurcation in the future of Transformer models: massive Real-valued models for general foundation models, and specialized Hypercomplex models for signal processing and edge-AI applications. The significant performance gains on MusicNet (Complex) and the parameter efficiency on SNLI (Quaternion) validate the hypothesis that aligning the network’s algebra with the data’s intrinsic structure—whether it be the phase of a sound wave or the multi-faceted nature of a word—yields tangible benefits in accuracy and efficiency.

References

- [1] Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” *ResearchGate*, accessed November 25, 2025.
- [2] Yang et al., “Complex Transformer: A Framework for Modeling Complex-Valued Sequence.” *ResearchGate/arXiv*, accessed November 25, 2025.
- [3] Tay et al., “Lightweight and Efficient Neural Natural Language Processing with Quaternion Networks.” *SMU Libraries*, accessed November 25, 2025.